The Lucas Critique in ML: Model Notes

An Intuitive Guide to Every Modelling and Estimation Choice

4815L — D200 Machine Learning in Economics

Purpose of this document. This note walks through every modelling and estimation choice in the project from first principles. The goal is intuition: why was this approach chosen, what is the algorithm actually *doing*, and how does it relate to the others? Mathematics is kept to the minimum needed to be precise.

Contents

1 The Data-Generating Process (DGP) 3
- 1.1 What it is . . . 3
- 1.2 Why simulate? . . . 3
- 1.3 The structural break (Lucas shift) . . . 3

2 Evaluation Framework 3
- 2.1 The Lucas Sensitivity Ratio (LSR) . . . 4
- 2.2 Structural break tests . . . 4

3 Model 1: AR(2) and ARMA(2,1) Baselines 4
- 3.1 What they are . . . 4
- 3.2 How they are estimated . . . 5
- 3.3 Why include them? . . . 5

4 Model 2: Markov Switching Regression (MSM) 5
- 4.1 What it is . . . 5
- 4.2 How it is estimated: the Hamilton EM algorithm . . . 5
- 4.3 Why MSM has the worst Lucas performance despite correct specification . . . 6

5 Model 3: Threshold Autoregression (TAR) 6
- 5.1 What it is . . . 6
- 5.2 How it is estimated . . . 6
- 5.3 Why TAR is relatively Lucas-stable . . . 7

6 Model 4: Hidden Markov Model + Ridge (HMM) 7
- 6.1 What it is . . . 7
- 6.2 How it is estimated: Baum-Welch EM . . . 7
- 6.3 The Viterbi algorithm . . . 8
- 6.4 Why HMM is the most Lucas-stable model . . . 8

7 Model 5: Mixture of Experts (MoE) 8
- 7.1 What it is . . . 8
- 7.2 How it is estimated: EM with soft responsibilities . . . 9
- 7.3 Difference from HMM . . . 9

8 Model 6: ML Regime (XGB) 9
- 8.1 What it is . . . 9
- 8.2 How XGBoost works . . . 10
- 8.3 Why XGB is the most Lucas-vulnerable model . . . 10

9 Model 7: Markov Switching Neural Network (MSNN) **10**
9.1 What it is 10
9.2 The MLP (Multi-Layer Perceptron) 11
9.3 How it is estimated: EM with Adam 11
9.4 The MSNN's paradox 12

10 Model 8: Model Average (Ensemble) **12**
10.1 What it is 12
10.2 Why it works 12

11 Feature Engineering **13**

12 Summary: How Each Model Answers the Fundamental Question **14**

The Data-Generating Process (DGP)

What it is

The simulated data comes from a **Markov-switching AR(1)** process:

$$y_t = \mu_{s_t} + \phi_{s_t}\, y_{t-1} + \sigma_{s_t}\, \varepsilon_t, \qquad \varepsilon_t \overset{\text{iid}}{\sim} \mathcal{N}(0,1), \qquad s_t \in \{0,1\}$$

The regime s_t switches between 0 (expansion) and 1 (recession) according to a Markov chain: the probability of staying in regime 0 is $P_{00} = 0.95$ and of staying in regime 1 is $P_{11} = 0.90$. Think of y_t as something like monthly GDP growth: it follows an AR(1) in normal times, but with a different mean, persistence, and volatility during recessions.

> **Intuition.** The economy flips between two "modes" (expansion and recession). Within each mode the series behaves like a standard autoregression, but the parameters — mean, persistence, volatility — are different in each mode. The *regime identity* s_t is **latent**: you can never directly observe whether you are in regime 0 or 1. All you see is the time series y_t. Every model in this project tries to infer s_t from y_t and use that inference to forecast.

Why simulate?

With simulated data the true s_t is known. This lets us measure how accurately each model *recovers the regimes* (regime accuracy, Adjusted Rand Index), not just how well it forecasts. We can also control the structural break exactly: we know the true parameters before and after, so we know how hard the problem is.

The structural break (Lucas shift)

After the training period, the DGP parameters change. The regime labels stay the same (still 0 and 1) but the mean, volatility, and transition probabilities shift. Models trained on the old parameters must now forecast under the new ones without being told anything changed. This is the Lucas Critique in action.

Two shift scenarios are used:

Parameter	Pre-break	Mild post	Severe post	Effect
μ_0 (expansion mean)	0.5	0.2	-0.5	Lower mean growth
μ_1 (recession mean)	-1.5	-1.0	-3.0	Deeper recessions
σ_0 (expansion vol)	1.0	1.3	2.0	More volatile
σ_1 (recession vol)	2.5	3.0	4.5	Much more volatile
P_{00} (expansion persist.)	0.95	0.90	0.80	Shorter expansions

The "mild" shift resembles a gradual policy tightening. The "severe" shift resembles the Volcker disinflation: a fundamental change in regime transition dynamics on top of parameter changes.

Evaluation Framework

The Lucas Sensitivity Ratio (LSR)

All models are trained on pre-break data only, then frozen. The LSR compares post-break forecast accuracy to pre-break accuracy:

$$\text{LSR} = \frac{\text{RMSE}_{\text{post}}}{\text{RMSE}_{\text{pre}}}$$

A model with LSR > 1 performs worse after the break. LSR $= 1$ means no degradation. LSR < 1 (which occurs for the Great Moderation break) means the model actually *improves* after the break because post-break data is less noisy.

> **Important nuance.** LSR is a *relative* metric. A model with very low pre-break RMSE (because it memorises training data) will mechanically have a high LSR, even if its absolute post-break RMSE is reasonable. The XGB model is the clearest example: pre-break RMSE of 0.40 produces LSR = 8.43, even though the post-break RMSE of 3.34 is not the worst in absolute terms.

Structural break tests

Two formal tests are applied to detect whether a break occurred at all.

Chow test. Fits a single regression over the full sample and two separate regressions (pre- and post-break). If the two sub-sample regressions fit much better than the pooled one, the parameters must have shifted. The F-statistic compares the reduction in residual sum of squares.

> **Intuition.** Imagine fitting a straight line through two clouds of points and then fitting two separate lines, one for each cloud. If the two-line fit is dramatically better, the clouds must come from different underlying relationships. The Chow test formalises this comparison.

CUSUM test. Computes the cumulative sum of prediction residuals over time. Under parameter stability, the cumulative sum should wander randomly around zero. A systematic drift away from zero signals that something has changed. The test plots a running total and checks whether it strays outside 5% significance bands.

The Chow test detects *parameter* shifts in an AR model. It does not detect a shift in *transition dynamics* alone — which is why it correctly detects the Great Moderation (a variance shift) but fails to detect the Volcker break (primarily a regime dynamics shift).

Model 1: AR(2) and ARMA(2,1) Baselines

What they are

The **AR(2)** model says the current value depends linearly on the two most recent values plus noise:

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \varepsilon_t$$

The **ARMA(2,1)** adds a moving average term: the current error also depends on last period's error:

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \varepsilon_t + \theta\, \varepsilon_{t-1}$$

How they are estimated

AR(2) is estimated by OLS: just regress y_t on y_{t-1} and y_{t-2}. ARMA is estimated by maximum likelihood, using a Kalman filter to compute the likelihood of the observed series given the parameters, then optimising over (ϕ_1, ϕ_2, θ).

Why include them?

These are the *benchmark*. They have no regime-switching at all. Their purpose is to ask: does any of the complexity of regime-switching actually help? If regime-switching models do not outperform a simple AR(2), the extra complexity is not justified.

> **Intuition.** AR(2) is essentially asking: "given where I was in the last two periods, where am I likely to be now?" It treats the economy as a single, stable linear process. The Lucas Critique predicts it will degrade after a structural break — and it does, with LSR around 1.66 for the mild shift. But because it fits the pre-break data modestly (RMSE ≈ 1.63), it does not have a large denominator inflating its LSR.

Model 2: Markov Switching Regression (MSM)

What it is

The MSM is Hamilton's (1989) classical regime-switching model. It assumes the data actually *comes from* the Markov-switching DGP we described above:

$$y_t = \mu_{s_t} + \phi_{s_t}\, y_{t-1} + \sigma_{s_t}\, \varepsilon_t$$

where s_t follows a two-state Markov chain. This is the *correctly specified* model for our simulated data: the DGP is MS-AR(1), and so is the MSM.

How it is estimated: the Hamilton EM algorithm

The challenge is that s_t is unobserved. We cannot simply OLS-regress y_t on y_{t-1} because we do not know which regime's parameters to use at each t.

The solution is the **EM (Expectation-Maximisation) algorithm**, a general approach for estimating models with missing data. For the MSM:

E-step (Hamilton filter): infer regimes from data. Given current parameter guesses $(\hat{\mu}_k, \hat{\phi}_k, \hat{\sigma}_k, \hat{P})$, compute the probability of being in each regime at each time step. This is done recursively: the probability of being in state k at time t is proportional to

1. How likely the observed y_t is under regime k's parameters (the Gaussian likelihood $\mathcal{N}(y_t;\, \hat{\mu}_k + \hat{\phi}_k y_{t-1},\, \hat{\sigma}_k^2)$).
2. The probability of *transitioning into* regime k from wherever we were last period (the transition probability from the Markov chain).

Running this forward through the data gives "filtered" regime probabilities. Running a backward smoother then refines these using future data.

M-step: re-estimate parameters given inferred regimes. Using the regime probabilities from the E-step as soft weights, re-estimate $(\mu_k, \phi_k, \sigma_k)$ by weighted least squares, and re-estimate P by counting weighted transition frequencies.

Alternate E-step and M-step until convergence (parameters stop changing).

> **Intuition.** Imagine trying to learn two different linear regression models from data where you do not know which observations belong to which model. EM solves this by iterating: first *guess* which model each point belongs to (E-step), then *refit* the models using those guesses (M-step). The Hamilton filter is the E-step for Markov-switching models: it propagates regime probabilities forward through time, accounting for the Markov transition structure.
> Think of the filter as a "belief update": at each time step, your prior belief about the regime is updated by how well the new observation fits under each regime's model. High volatility in y_t raises the probability of being in the high-variance recession regime.

Why MSM has the worst Lucas performance despite correct specification

This is one of the most striking findings. The MSM is *exactly* the right model for the simulated DGP, yet its post-break RMSE is the worst of all models (5.97 mild, 11.96 severe).

The reason is the Hamilton filter. It uses the *estimated* emission densities $\mathcal{N}(y_t;\, \hat{\mu}_k + \hat{\phi}_k y_{t-1},\, \hat{\sigma}_k^2)$ to classify regimes. After the break, μ_k and σ_k have changed, so the filter's emission densities are wrong. The filter confidently assigns the *wrong* regime because it is looking for the old distribution. Post-break regime accuracy collapses from 75% to 20% (effectively random). Once the regime classification is wrong, the regime-specific forecasts become completely wrong too.

> **Key lesson.** Correct pre-break specification actually *increases* Lucas vulnerability. A model that commits strongly to the true pre-break structure has further to fall when that structure changes. The MSM's precise identification of regimes is a liability, not an asset, when the regime structure itself shifts.

Model 3: Threshold Autoregression (TAR)

What it is

The TAR model divides the data into two regimes based on whether the lagged value of y exceeds a threshold τ:

$$y_t = \begin{cases} \hat{\beta}_0^\top x_t & \text{if } y_{t-1} \leq \hat{\tau} \\ \hat{\beta}_1^\top x_t & \text{if } y_{t-1} > \hat{\tau} \end{cases}$$

The threshold $\hat{\tau}$ is chosen by grid search: try every candidate threshold (at quantiles from 15% to 85% of the distribution of y_{t-1}), and pick the one that minimises the total sum of squared residuals across both regimes. Features x_t include y_{t-1}, rolling means, and covariates.

How it is estimated

1. **Grid search for $\hat{\tau}$:** for each candidate threshold, split the data and fit OLS/Ridge in each partition. Compute total SSR. Pick the threshold that gives the lowest SSR.
2. **Re-fit Ridge regressors** on each partition using the chosen $\hat{\tau}$.

> **Intuition.** The TAR says: "the economy behaves differently when growth was recently weak versus recently strong." The threshold is the boundary between these two states. If y_{t-1} was below $\hat{\tau}$ (bad times recently), use one regression; if above (good times recently), use another. The grid search is asking: what value of last period's output best separates "expansion behaviour" from "recession behaviour"? It tries 50 candidate split-points and picks the one that produces the cleanest partition.

Why TAR is relatively Lucas-stable

The threshold $\hat{\tau}$ is a hard cutoff in observable data (y_{t-1}), not in a latent state that depends on fitted emission densities. After the break, the TAR's regime classification is still driven by observable y_{t-1} relative to a fixed threshold. The threshold may not perfectly separate regimes any more, but it does not fail as catastrophically as the MSM's Gaussian filter. LSR = 1.64 (mild), 2.78 (severe) — among the best.

Model 4: Hidden Markov Model + Ridge (HMM)

What it is

The HMM is a **two-stage** pipeline:

1. A **Gaussian HMM** infers latent regimes from an enriched feature vector.
2. A **Ridge regressor** is fitted separately for each inferred regime.

The HMM is estimated on the *observation matrix*:

$$\mathbf{O}_t = \left[y_t,\ y_{t-1},\ \bar{y}_t^{(5)},\ s_t^{(5)},\ \bar{y}_t^{(20)}\right]^\top$$

where $\bar{y}^{(5)}$ is the 5-period rolling mean, $s^{(5)}$ is the rolling standard deviation, and $\bar{y}^{(20)}$ is the 20-period rolling mean.

How it is estimated: Baum-Welch EM

The HMM uses the same EM idea as the MSM, but the emission model is a multivariate Gaussian over $\mathbf{O}_t$ rather than a univariate one over y_t alone.

E-step (Baum-Welch forward-backward algorithm):

- *Forward pass*: compute $\alpha_t(k) = P(s_t = k,\ \mathbf{O}_{1:t})$ recursively, using the multivariate Gaussian emission probability of $\mathbf{O}_t$.
- *Backward pass*: compute $\beta_t(k) = P(\mathbf{O}_{t+1:T} \mid s_t = k)$.
- *Smoothed posteriors*: $\gamma_t(k) = P(s_t = k \mid \mathbf{O}_{1:T}) \propto \alpha_t(k)\,\beta_t(k)$.
- *Joint posteriors*: $\xi_t(j,k) = P(s_t = j,\ s_{t+1} = k \mid \mathbf{O}_{1:T})$.

M-step: update emission Gaussians (means and covariances) using the soft regime weights $\gamma_t(k)$, and update the transition matrix from $\xi_t(j,k)$.

After convergence, **Viterbi decoding** finds the single most probable state sequence $s^*_{1:T}$, which is used to partition the training data for fitting the Ridge regressors.

At forecast time, a *causal forward filter* (no backward pass) identifies the current regime from past observations only.

The Viterbi algorithm

Viterbi is *dynamic programming* applied to hidden Markov models. It finds the globally best path through the hidden states rather than the best local state at each step.

1. **Initialise:** $\log \delta_1(k) = \log \pi_k + \log b_k(\mathbf{O}_1)$.
2. **Recurse:** $\log \delta_t(k) = \max_j[\log \delta_{t-1}(j) + \log A_{jk}] + \log b_k(\mathbf{O}_t)$.
   Store backpointer $\psi_t(k) = \arg\max_j[\log \delta_{t-1}(j) + \log A_{jk}]$.
3. **Backtrack:** start at $s_T^* = \arg\max_k \log \delta_T(k)$, then follow backpointers backwards: $s_t^* = \psi_{t+1}(s_{t+1}^*)$.

Log-space is used throughout to avoid numerical underflow when multiplying many small probabilities.

> **Intuition.** Imagine a map where each column is a time step and each row is a possible regime. Viterbi fills in the map column by column: the entry $\delta_t(k)$ records the probability of the best *path* that ends at regime k at time t (not just the local probability). When it reaches the final column, it picks the best endpoint and traces back the entire path.
> The key difference from the forward filter: Viterbi gives a **hard** assignment (one definite regime per time step). The forward-backward smoother gives **soft** probabilities ($\gamma_t(k) \in (0, 1)$). Both are $O(K^2T)$ — quadratic in regimes, linear in time.

Why HMM is the most Lucas-stable model

1. **Richer observation matrix.** The HMM sees not just y_t but also 5- and 20-period rolling statistics. These slow-moving signals carry information about the regime that does not change instantly after a parameter shift, giving the forward filter more stable inputs.
2. **Soft regime assignments.** The forward filter outputs a probability distribution over regimes at each step, not a hard decision. After the break, when the emission densities are slightly wrong, the filter spreads probability mass across regimes rather than committing catastrophically to the wrong one.
3. **Decoupled regime detection and forecasting.** The HMM identifies regimes independently of what the Ridge regressors predict. Even if the Ridge weights are slightly wrong post-break, the regime classification is not contaminated.
4. **Ridge regularisation.** Ridge shrinks regression weights towards zero, reducing overfitting to the pre-break regime-specific patterns.

Model 5: Mixture of Experts (MoE)

What it is

The Mixture of Experts is a **soft routing** model: rather than assigning each observation to exactly one regime, it blends the predictions of K "expert" regressors using a **gating network**

that assigns weights to each expert:

$$\hat{y}_t = \sum_{k=1}^{K} \pi_k(x_t) \cdot \hat{\beta}_k^\top x_t$$

The gating weights $\pi_k(x_t) = P(\text{expert} = k \mid x_t)$ come from a logistic regression. The experts are Ridge regressors.

How it is estimated: EM with soft responsibilities

E-step: given current parameters, compute *responsibilities* (which expert should "own" each observation):

$$r_{ik} = \frac{\pi_k(x_i) \cdot \mathcal{N}(y_i;\, \hat{\beta}_k^\top x_i,\, \hat{\sigma}_k^2)}{\sum_j \pi_j(x_i) \cdot \mathcal{N}(y_i;\, \hat{\beta}_j^\top x_i,\, \hat{\sigma}_j^2)}$$

The responsibility r_{ik} is high for expert k when: (a) the gating network assigns high probability to k at x_i, *and* (b) expert k's prediction is close to the actual y_i.

M-step: re-estimate the gating network (logistic regression on $(x_i, \arg\max_k r_{ik})$) and re-estimate each Ridge expert using the responsibilities as sample weights.

> **Intuition.** Think of three people (experts) each proposing a forecast, and a coordinator (the gating network) deciding how much to trust each person based on the current economic conditions. Each person learns only from the situations they are trusted to handle. The EM algorithm refines both the experts and the coordinator simultaneously: if an expert is more accurate, it gets more responsibility, which makes its weights better, which makes it more accurate.
> Unlike TAR or MSM, *every* expert contributes to every forecast proportionally to its gating weight. This "soft" blending provides partial regularisation: a misclassified observation in the wrong regime does not completely corrupt the forecast, it just reduces the correct expert's weight.

Difference from HMM

The MoE has no Markov chain. Each time step's regime probability is determined solely by the current x_t, with no memory of the previous regime. The HMM tracks how regimes evolve over time via the transition matrix; the MoE treats each step independently. This makes MoE faster to estimate but means it cannot learn that "recessions last 10 periods on average."

Model 6: ML Regime (XGB)

What it is

The ML Regime model uses a three-stage pipeline:

1. **KMeans clustering**: cluster training observations into $K = 2$ groups using lags, rolling statistics, and covariates. These clusters serve as pseudo-regime labels. No economics assumed — purely data-driven.

2. **XGBClassifier**: train a gradient-boosted classifier to reproduce those cluster labels from the same feature set. This gives a smooth, interpretable mapping from features to regime probabilities.

3. **Per-regime XGBRegressors**: train one XGBoost regressor on each regime's subset of training data.

How XGBoost works

XGBoost builds an **ensemble of decision trees** in sequence. Each tree tries to correct the errors of all previous trees. The prediction is the sum of all tree outputs:

$$\hat{y}_t = \sum_{m=1}^{M} f_m(x_t)$$

where each f_m is a small decision tree. Trees are added greedily: at each step, find the tree f_m that most reduces the loss on the current residuals.

Decision trees work by splitting the feature space: "if rolling std > 2.1 AND lag-1 < -0.5, predict -1.3; else predict $+0.4$". XGBoost's output is therefore a **piecewise-constant function**: the feature space is divided into rectangular regions, each with a constant prediction.

> **Intuition.** Imagine drawing a very detailed decision flowchart for predicting output: "If the economy was below -0.5 last month AND volatility was above 2.1, AND it rained in Des Moines..." With 200 trees and no restraint, XGBoost can carve the feature space into tiny boxes, each perfectly fitting the training observations inside it. This is the memorisation that causes its spectacular collapse after the structural break.
> The piecewise-constant nature is the key vulnerability. Once the DGP shifts, the box boundaries (learned from pre-break data) no longer correspond to meaningful divisions. But because the boxes are rigid, the model cannot interpolate between them — it just outputs the wrong pre-break average for each new observation that lands in an old box.

Why XGB is the most Lucas-vulnerable model

1. **Frozen KMeans boundaries.** The regime classifier uses cluster centroids fixed at training time. Post-break, the same boundaries systematically misclassify regimes.

2. **Piecewise-constant extrapolation.** Unlike polynomials or neural networks, tree predictions do not interpolate smoothly between training points. A post-break observation that falls in a region with no training data gets the pre-break average for that region — which may be arbitrarily wrong.

3. **Extreme in-sample fit.** Pre-break RMSE of 0.40 — about 4× lower than the next best model — reflects memorisation of the training data. This also inflates the LSR denominator, making the ratio look worse than it is in absolute terms.

Model 7: Markov Switching Neural Network (MSNN)

What it is

The MSNN combines:

- **A Markov chain** governing regime transitions (like the MSM).
- **Per-regime MLP experts** that produce forecasts (instead of linear regressions).

The data-generating assumption is:

$$y_t = f_{s_t}(x_t) + \sigma_{s_t}\,\varepsilon_t, \qquad s_t \mid s_{t-1} \sim P$$

where each f_k is a small feedforward neural network.

The MLP (Multi-Layer Perceptron)

Each expert is an MLP with architecture $n_{\text{feat}} \to 32 \to 16 \to 1$:

- **Input layer**: the feature vector x_t (lags, rolling stats, covariates).
- **Hidden layer 1 (32 neurons)**: each neuron computes $\text{ReLU}(w^\top x + b)$. ReLU means "zero if negative, identity if positive" — it introduces nonlinearity while remaining computationally simple.
- **Hidden layer 2 (16 neurons)**: same structure.
- **Output layer (1 neuron)**: linear combination of the 16 hidden activations, producing a scalar forecast.

> **Intuition.** The MLP can represent arbitrarily complex nonlinear functions, given enough neurons. Think of it as a very flexible curve-fitter: rather than assuming y_t depends *linearly* on lagged values, the MLP can learn any smooth relationship from the training data. Mathematically, a two-layer ReLU network is a piecewise-*linear* function of the inputs (unlike XGBoost's piecewise-*constant* output), which allows it to interpolate between training points rather than jumping to flat plateaus.

How it is estimated: EM with Adam

The MSNN uses the same EM structure as the MSM and HMM, but the M-step updates MLPs via gradient descent instead of closed-form weighted OLS.

E-step (Hamilton forward-backward filter): Same as the MSM, but the emission probability $P(y_t \mid s_t = k)$ is computed from the MLP's prediction: $\mathcal{N}(y_t;\, f_k(x_t),\, \sigma_k^2)$.

Run the forward filter:

$$\log \alpha_t(k) = \log \sum_j e^{\log \alpha_{t-1}(j) + \log P_{jk}} + \log p(y_t \mid s_t = k)$$

Then the backward filter, and combine to get smoothed posteriors $\gamma_t(k)$.

M-step (Adam for MLP, analytical for transition matrix):

- **Transition matrix**: $\hat{P}_{jk} \propto \sum_t \xi_t(j,k)$. This is just counting weighted transitions — no gradient needed.
- **MLP experts**: minimise the weighted loss $\mathcal{L}_k = \sum_t \gamma_t(k)\,(y_t - f_k(x_t))^2$ using the **Adam** optimiser. Adam adjusts the learning rate individually for each weight based on the recent history of gradients.

Adam intuition. Imagine walking downhill (gradient descent) to find the lowest point in a landscape. Vanilla gradient descent takes equal-sized steps in the direction of steepest descent. Adam keeps a running average of the gradient direction (*momentum*) and a running average of the squared gradient magnitude (*adaptive scaling*). This means it takes bigger steps when the gradient has been consistently pointing in one direction, and smaller steps when gradients have been noisy. In practice this converges much faster and more reliably than plain gradient descent for neural networks.

The MSNN's paradox

The MSNN has:

- **Worst regime identification**: in-sample regime accuracy $\approx 51\%$ (near random). The EM collapsed to a single dominant regime; the Markov chain stopped being meaningful.
- **Best absolute post-break RMSE**: 2.493 (mild), 3.391 (severe) — the lowest of all models in every experiment.

How? Because the MLP experts are smooth universal approximators. Even if the Markov structure has collapsed, the MLP can learn a good overall nonlinear mapping from x_t to y_t. Post-break, the MLP *extrapolates smoothly* outside the training distribution, whereas XGBoost's step functions return stale pre-break box averages.

Why smooth extrapolation is important for the Lucas Critique. After a structural break, test observations lie in a region of feature space that training data only partially covers. Smooth functions (MLPs) extrapolate by continuing the learned smooth relationship into this new region. Piecewise- constant functions (XGBoost trees) return the average of the nearest training box, which may be far from the new distribution. This is why the MSNN wins on absolute accuracy despite its poor in-sample regime identification.

Model 8: Model Average (Ensemble)

What it is

The model average simply takes the **equal-weight mean** of all other models' predictions at each time step:

$$\hat{y}_t^{\text{avg}} = \frac{1}{N} \sum_{n=1}^{N} \hat{y}_t^{(n)}$$

No additional estimation is needed — it is computed at prediction time only.

Why it works

The theoretical justification comes from the bias-variance trade-off. If individual models have correlated errors, averaging reduces variance but does not hurt bias much. But more relevantly here: the forecast risk decomposition shows that the model class $\mathcal{F}$ contributes an *approximation error* term. Averaging across models with different functional forms effectively expands $\mathcal{F}$, reducing the approximation error component.

Intuition. If one expert is wrong in one direction and another is wrong in the other direction, averaging them gets you closer to the truth. More formally: even if no single model has the right functional form for the new regime, the average of models with diverse functional forms is likely to be closer to the truth than any individual model. In 2 out of 4 experiments, the ensemble produces both a lower LSR and lower absolute post-break RMSE than most individual models.

Feature Engineering

All models (except AR/ARMA and the raw MSM) use the same feature set:

Feature	Formula	Intuition
`y_lag1`	y_{t-1}	Most recent value
`y_lag2`	y_{t-2}	Two-period lag
`roll_mean_5`	$\bar{y}_t^{(5)} = \frac{1}{5}\sum_{i=0}^{4} y_{t-i}$	Short-run trend
`roll_std_5`	$s_t^{(5)}$	Short-run volatility
`roll_mean_20`	$\bar{y}_t^{(20)}$	Long-run trend
`roll_std_20`	$s_t^{(20)}$	Long-run volatility
`exog_0`	EFFR (simulated) / EFFR (real)	Interest rate level
`exog_1`	Simulated / UNRATE	Labour market

The rolling statistics are particularly important for the HMM. The HMM observation matrix uses $[\bar{y}^{(5)}, s^{(5)}, \bar{y}^{(20)}]$ rather than raw y values, because regime transitions are more reliably detected from slow-moving summary statistics than from a single noisy observation.

Summary: How Each Model Answers the Fundamental Question

Model	How does it find regimes?	How does it forecast?	Lucas vulnerability
AR(2) / ARMA	Doesn't — one global model	Linear in lags	Moderate: no regime assumptions to break
MSM	Hamilton EM filter on y_t	Regime-probability-weighted AR	High: emission densities wrong post-break; filter collapses
TAR	Grid-search threshold in y_{t-1}	Per-regime Ridge (hard split)	Low-moderate: observable threshold, no latent state to misidentify
HMM	Baum-Welch EM on rolling-feature matrix	Per-regime Ridge (soft split)	Low: soft posteriors, rich observation matrix
MoE	Logistic gating via EM	Responsibility-weighted Ridge	Moderate: soft but no Markov memory
ML XGB	KMeans → XGB classifier	Per-regime XGBRegressor	Very high: frozen boundaries + piecewise constant
MSNN	Markov + MLP (EM)	Per-regime MLP	High LSR / Low abs. RMSE: MLP smooth extrapolation
Ensemble	—	Equal-weight average	Low: diversification reduces approximation error

The big picture. Every model in this project answers the same question: "given what I know about the economy's past, which *mode* is it currently in, and how should I forecast from that mode?"

The models differ in three key dimensions:

1. **How they find the mode** (hard threshold, probabilistic filter, clustering, EM with Markov chain).
2. **How flexible the forecast is** within each mode (OLS, Ridge, XGBoost trees, MLP).
3. **How much they commit to the pre-break mode structure** (MSM commits maximally; HMM commits softly; MSNN commits to functional form but not regime structure).

The central finding — flexible latent-state estimation beats flexible functional form — can be rephrased simply: it is better to remain uncertain about *which mode you are in* (HMM's soft posteriors) than to be very certain about *what function to use* within a mode (XGBoost's trees). After a structural break, the mode boundaries shift. Uncertain regime assignments partially absorb this; rigid functional forms cannot.